

Maximum Flows and Minimum Cuts

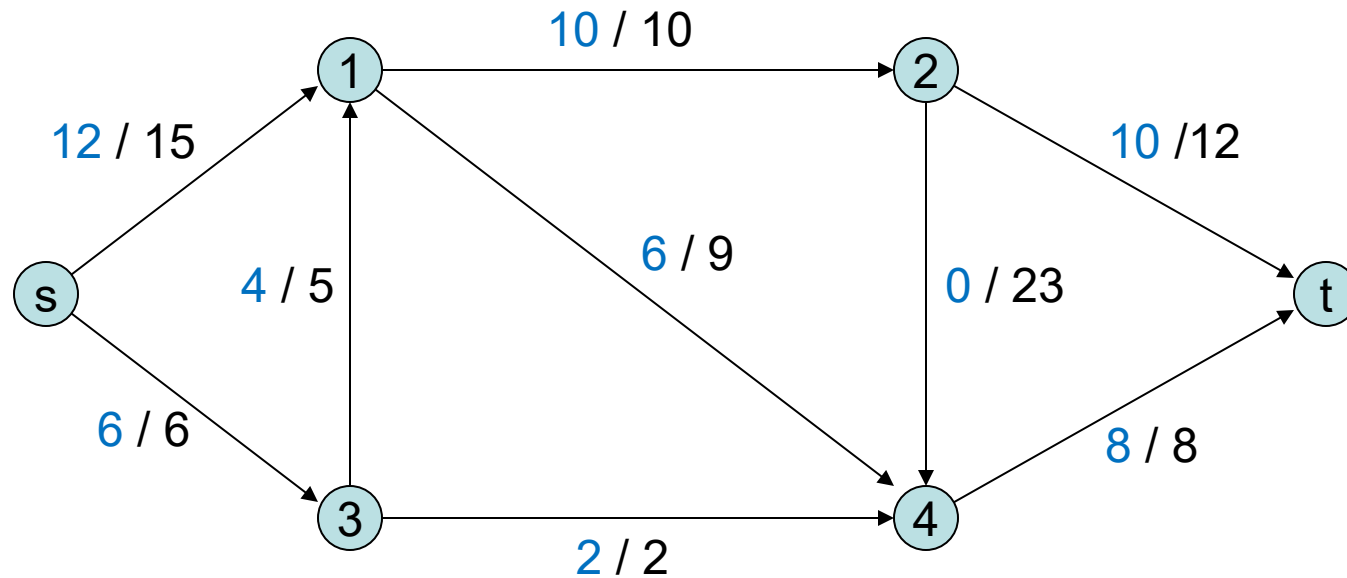
Design and Analysis of Algorithms

Brian C. Dean

**School of Computing
Clemson University**

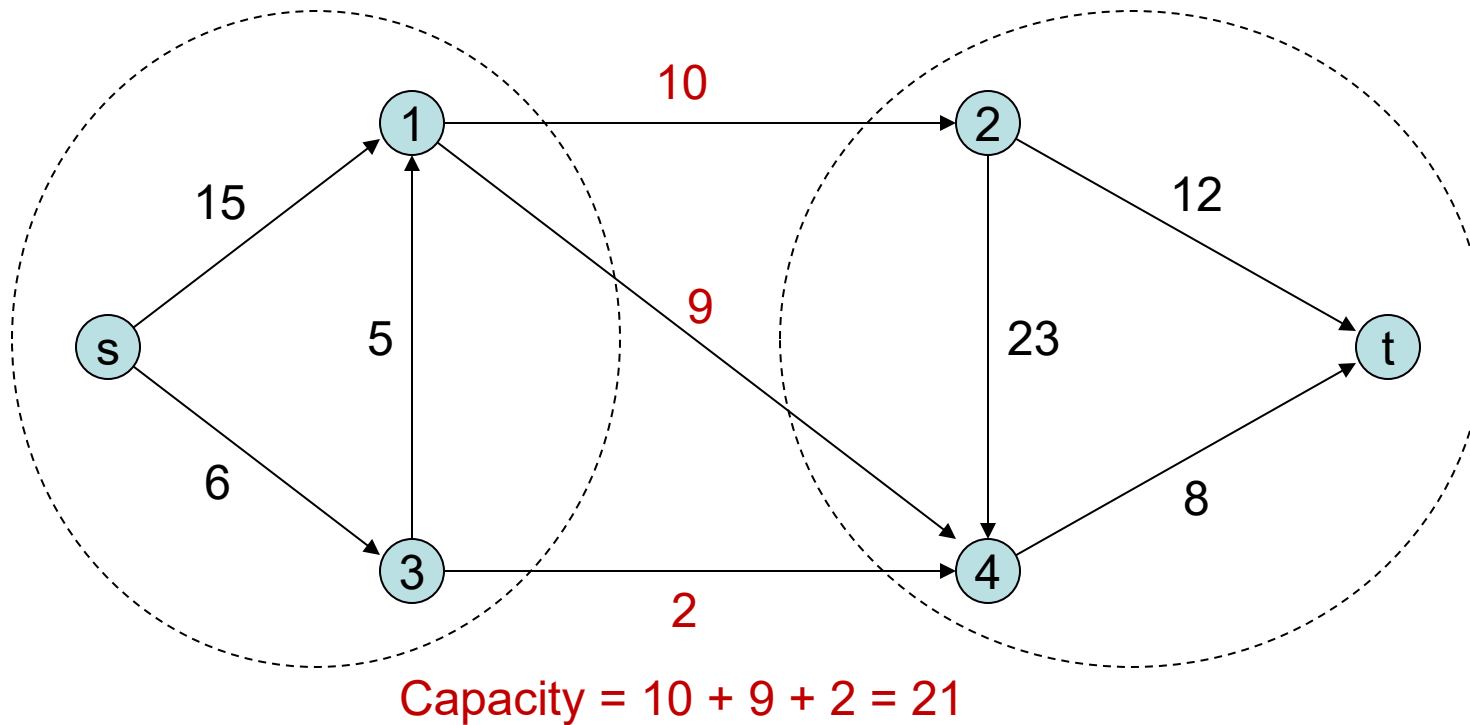


The Maximum Flow Problem



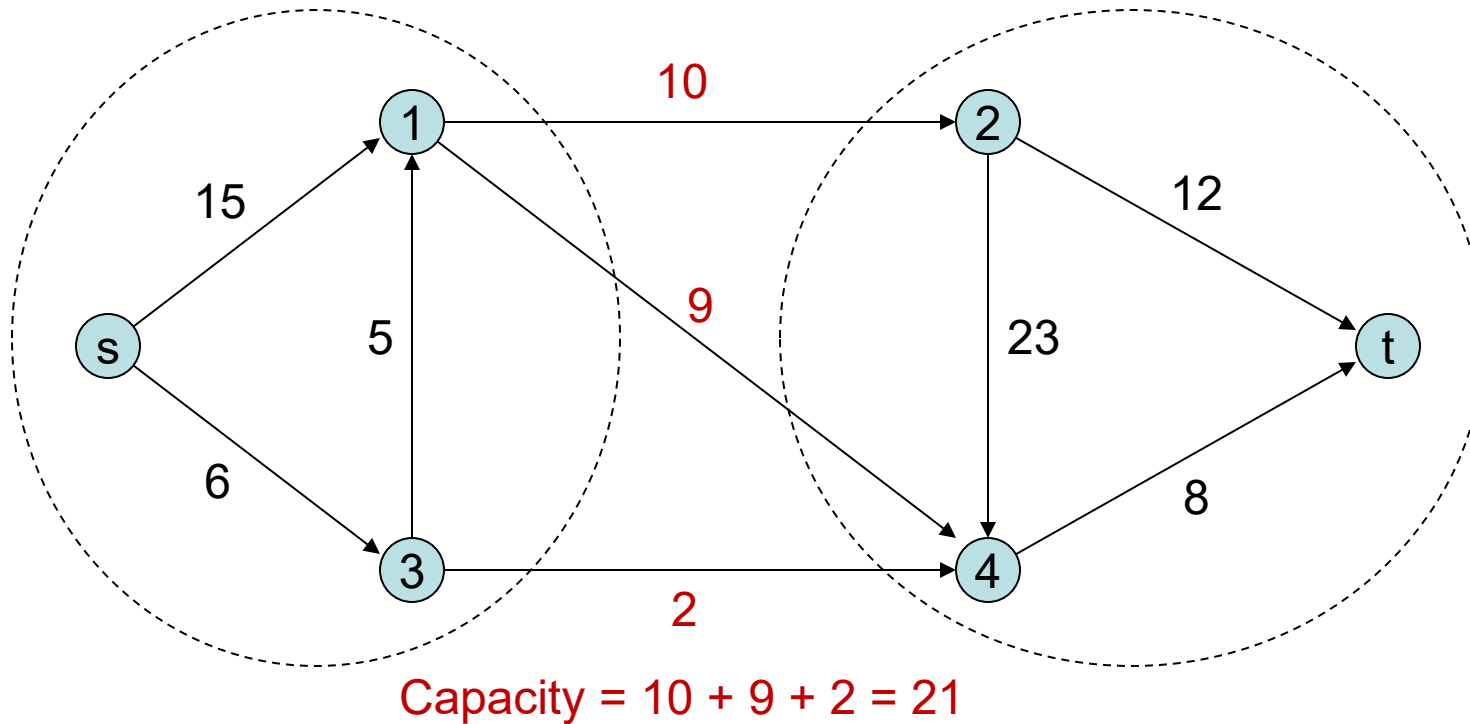
Given a directed graph with capacities on edges, what is the maximum amount of flow that can be shipped from a source node s to a sink node t ? **18 units**

s-t Cuts



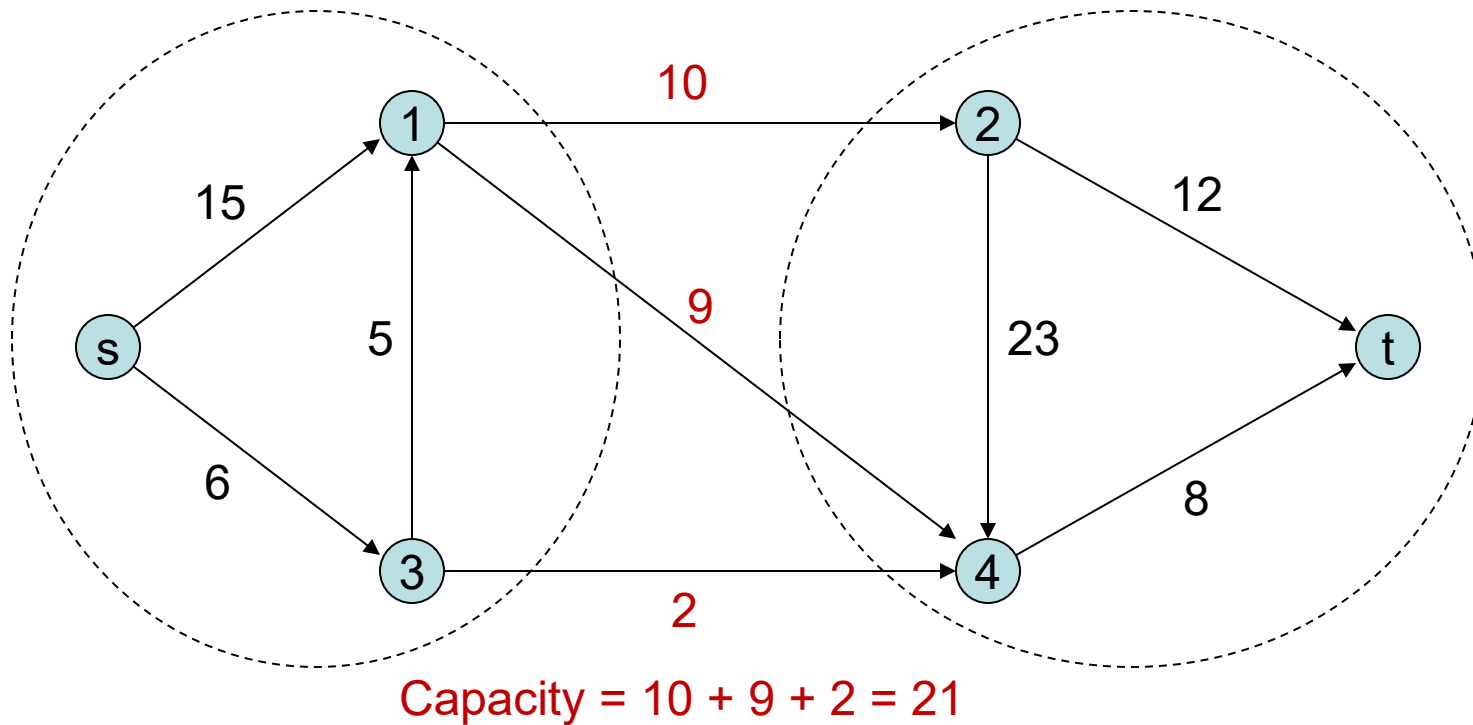
- An s-t cut is a partition of the nodes in our graph into two sets, one containing s and the other containing t.
- The capacity of a cut is the sum of the capacities of the edges crossing the cut in the $s \rightarrow t$ direction.

s-t Cuts



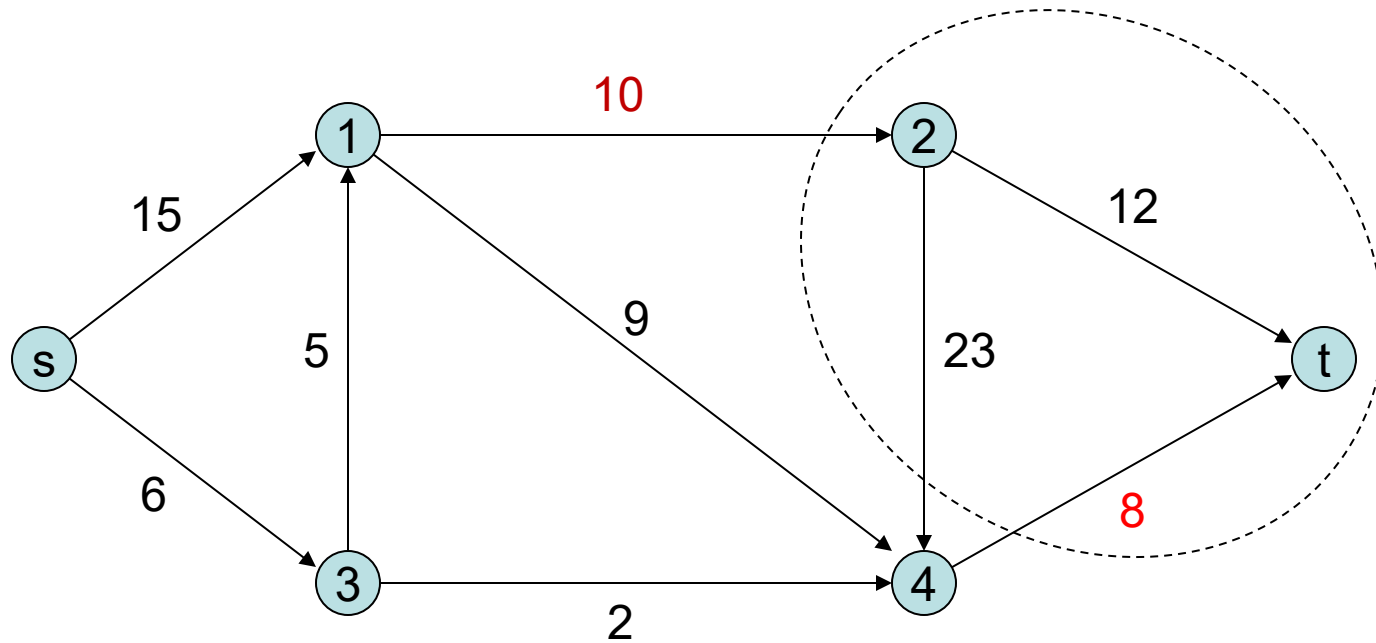
- **Easy observation:** The value of the maximum flow is at most the capacity of any cut.
- So: $\max \text{ s-t flow value } \leq \min \text{ s-t cut capacity }$

s-t Cuts



- **Easy observation:** The value of the maximum flow is at most the capacity of any cut.
- So: $\max \text{ s-t flow value} \leq \min \text{ s-t cut capacity}$
- **Famous result:** $\max \text{ s-t flow value} = \min \text{ s-t cut capacity}$.

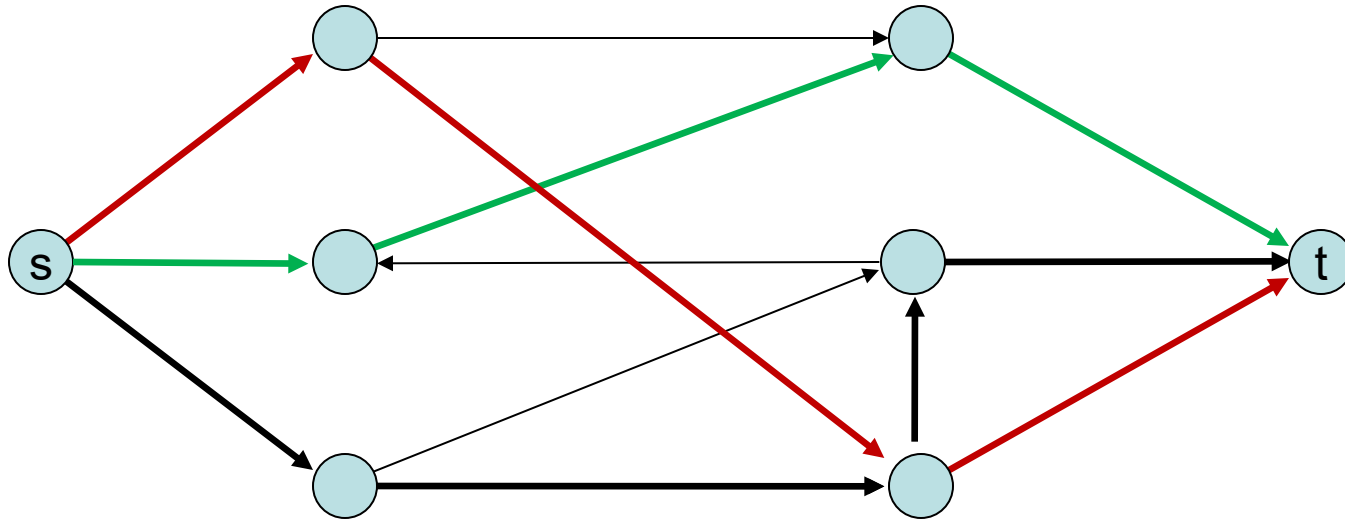
s-t Cuts



$$\text{Capacity} = 10 + 8 = 18$$

- **Easy observation:** The value of the maximum flow is at most the capacity of any cut.
- So: $\max \text{ s-t flow value} \leq \min \text{ s-t cut capacity}$
- **Famous result:** $\max \text{ s-t flow value} = \min \text{ s-t cut capacity}$.

Higher Levels of Connectivity



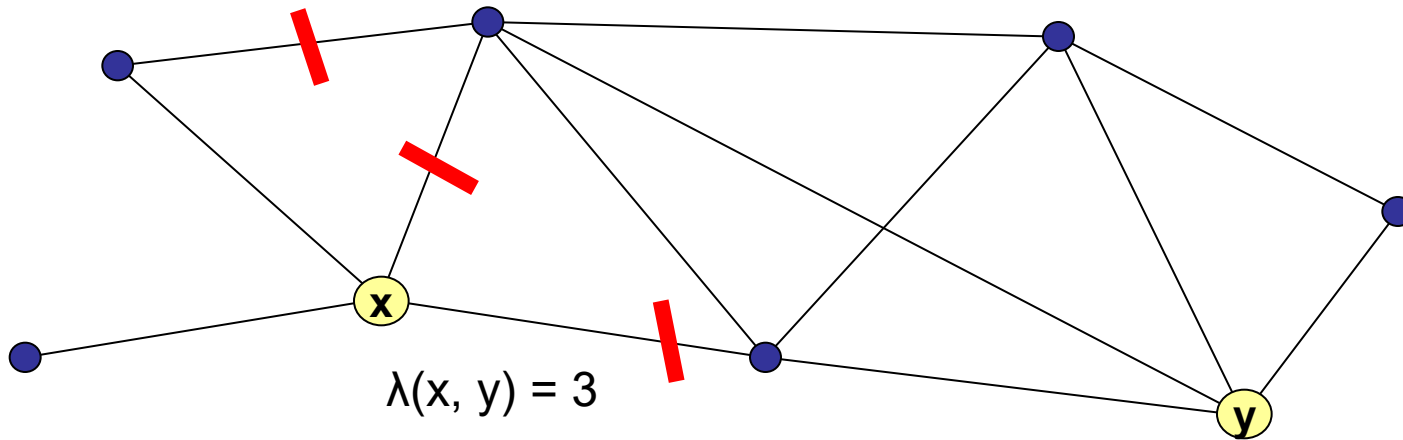
Maximum # of edge-disjoint
s-t paths

(path decomposition of a
max flow in a graph with
unit capacities)

= Minimum # of edges we
need to remove to separate
s and t.

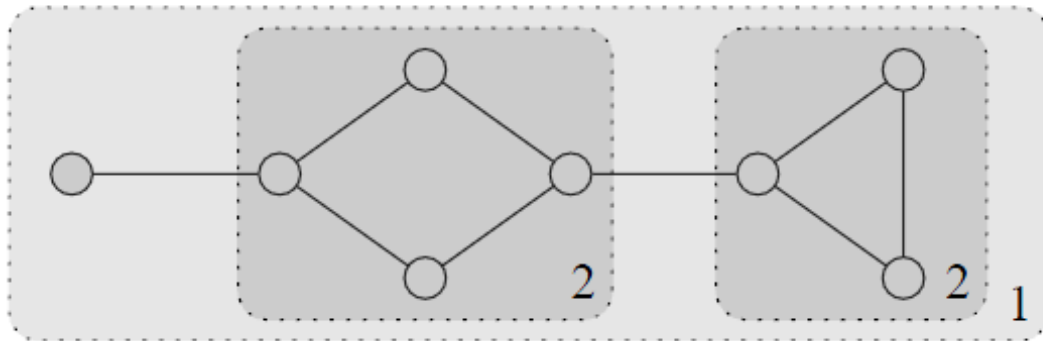
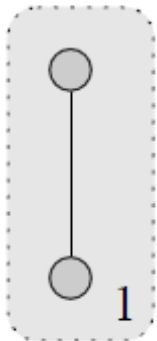
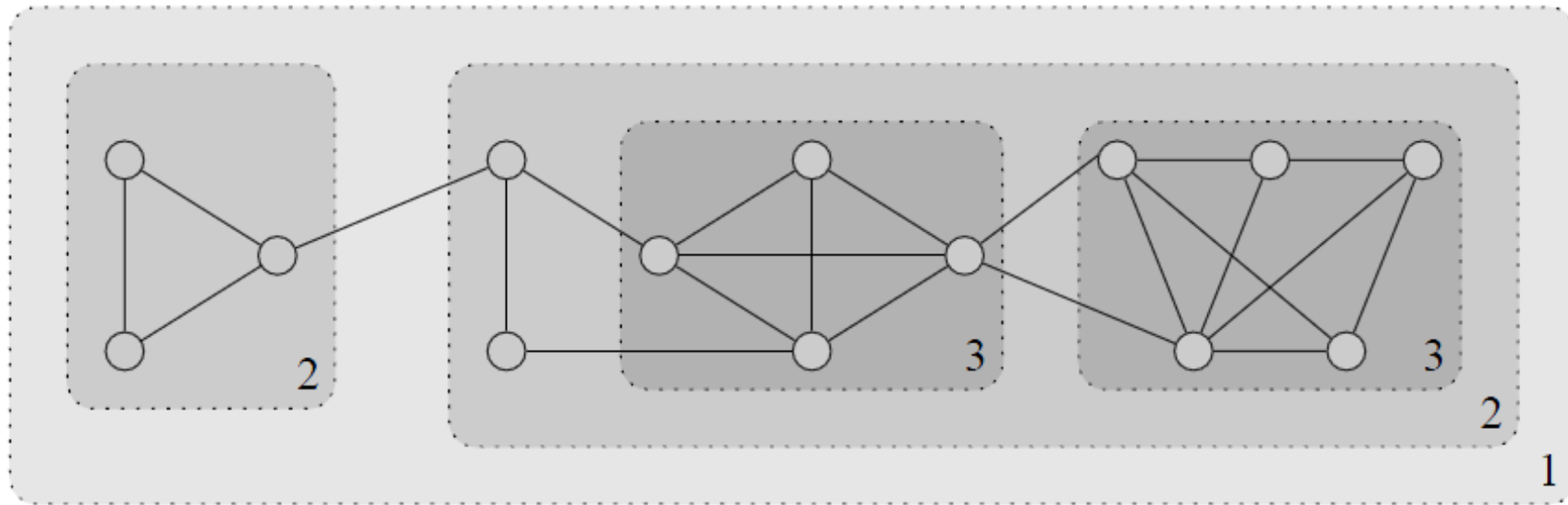
(minimum s-t cut)

Higher Levels of Connectivity



- Let $\lambda(x,y)$ denote the value of a unit-cap max flow from x to y .
- Equivalently, the minimum # of edge removals to separate x from y .
- This quantity is known as the edge connectivity between x and y .

The Hierarchical, Nested Structure of k -Edge-Connected Components



Calculating Max Flows

- Ford-Fulkerson algorithm: repeatedly push flow through s-t “augmenting paths”.
- When no augmenting path available, we’ve found a matching s-t flow and s-t cut, both of which therefore are optimal!

Calculating Max Flows

- Ford-Fulkerson algorithm: repeatedly push flow through s-t “augmenting paths”.
- Some possible running times:

Type of Aug Path:	Running Time per Augmentation	# Augmentations	Total Runtime
Arbitrary	$O(m)$, via DFS	$O(F)$ (F = max flow)	$O(mF)$
Shortest	$O(m)$, via BFS	$O(mn)$	$O(m^2n)$
Fattest	$O(m + n \log n)$, via Dijkstra	$O(m \log F)$	$O((m + n \log n)(m \log F))$

(in practice, things usually run much faster than running times would predict)